从街机厅到浏览器

六款单文件 Canvas 游戏的设计笔记与心路历程

TWO DESIGN LINEAGES · SIX SINGLE-FILE CANVAS GAMES

> 这份文档不讲代码怎么写。它讲的是：**我为什么先做了一款太空射击，又为什么把它拆成了六款**；我从 Space Invaders 到东方、从 Rogue 到 Hades 这两条设计血脉里，各自偷了什么；我在哪些地方想错了，错在哪一天被推翻，以及**推翻我的不是直觉，是我自己写的判据**。

> 文档末尾是那份技术解析（架构、算法、缺陷档案、判据自误案例）的摘要，以及完整的文件地图和版本时间线。

作品 六款单文件 HTML5 + Canvas 2D 游戏 + 一份初版存档

线上 kevin0521-wjw.github.io/canvas-games

规模 源码 17,498 行 / 约 758 KB，外部依赖 0，素材 0

验证 247 项判据，跑在真实浏览器里，6/6 款全绿

版本 v1.5.1 · 2026-11-10

本文档由 headless Edge 打印为 PDF

源文件：`games/docs/开发过程与借鉴分析.html`

目 录

CONTENTS

第 一 章 两条血脉

- 太空射击：从 Space Invaders 到「弹幕」是一种美学
- 肉鸽：从 Rogue 到 Hades
- 两条血脉的公共语法，以及我为什么做了六款

第 二 章 起点：第一款做砸了

- 星际防线 · 初版：我想做什么
- 「31 项断言全绿」的翻车现场
- 从这一跤里学到的东西

第 三 章 骨架：一个刻意的决定

- 八层结构
- 为什么一定要统一
- 一次分歧：同一题材做两条路线

第 四 章 六款，各走一条路

- 星际防线 v1 & v2：把参数抽出来，再把三套系统缝起来
- 符箓灵弹 · 巧阵：把「下压」做成可计算的墙
- 符箓灵弹 · 元素阵：把反应从倍率推到状态
- 星屑符咒：弹幕可读性，判据量不到「东西在哪」
- 深渊回响：房间即关卡
- 星轨 v2：缝合而不打架

第 五 章 难度观：我怎么从「削敌人」转向「加解法」

- v1.4.0：一次统一降难度
- v1.5.0：降过头了，同一条曲线不能跨品类照搬
- 最后我信什么

第 六 章 AI 观：让机器替我玩

- 为什么 AI 接管是「诚实的验收」
- AI 写不对的三种方式
- v1.5.1：AI 看不见新元素，比「AI 打不赢」更隐蔽的失败

第 七 章 验证观：全绿不等于对

- 判据自己错的十一种方式
- 「定义了却没人调用」「字段只写不读」

· 截图是判据的一部分

第 八 章 借鉴来源与取舍

第 九 章 交付与运行

第 十 章 附录 A　技术解析摘要

第 十一 章 附录 B　文件地图

第 十二 章 附录 C　版本时间线

第 十三 章 附录 D　参考资料与延伸观看

CHAPTER ONE

两条血脉

> 我做的六款游戏，只有两款是「肉鸽」，两款是「太空射击」，两款是「弹幕 / 元素反应」。但它们共享的东西，比品类差异大得多。在动手写第一行代码之前，我脑子里其实只有**两条设计血脉**：一条告诉我「弹幕应该被读懂，而不是被躲过」，另一条告诉我「失败应该是代价，而不是惩罚」。这两条血脉加起来，解释了我后来所有的取舍。

1.1 太空射击：从 Space Invaders 到「弹幕」是一种美学

太空射击的历史比电子游戏本身还长一点。公认的起点常被推到 1962 年 MIT 的 `Spacewar!`：两个圆点互相发射方块。再往前还有 WWII 末期的 `Wouter 't Hardt` 那套模拟海战的电子设备，但那已经不算「游戏」了。

1.1.1 1978 · Space Invaders：把「空间压迫感」写进了语法

Taito 的《太空侵略者》几乎只用了一个按键，却定下了此后半个世纪射击游戏的语法：**你移动，他们移动，你在他们走到你面前之前把他们打掉**。它的三个设计至今还活着：

① 屏幕即走廊

玩家被钉在底部，活动范围是一条水平线。这条限制本身就是难度来源。敌人没变强，是你无处可退。

② 击破加速

每消灭一个入侵者，剩下的就往下走快一点。残局自然变得更快更紧张，难度被写进了剩余敌人的速度里。

③ 可预期的节拍

敌群整体下压一格、玩家开一炮，都是固定节奏。玩家能进入「节奏」而不是「反应」状态，这是弹幕能成立的前提。

第三点尤其重要。**如果敌人的行动不可预期，屏幕上再多子弹也只是混乱。** Space Invaders 教会我的是：**公平感来自可读的规律，而不是来自弹量的减少**。

1.1.2 1981 · Galaga：敌人开始有「意图」

南梦宫的《蜂巢保卫战》引入了编队、俯冲、以及可以被拦截的「奖励机」。关键变化是**敌人从一个「阵列」变成了「一群有行为的个体」**：脱离编队单独俯冲的那只会脱离节奏，于是玩家必须判断「哪只已经脱离」，游戏第一次要求玩家**读意图**，不止于读位置。

1.1.3 1982 – 1987 · 卷轴时代：舞台开始移动

1981 年的《Defender》带来了横向卷轴与「保护人类」；1982 年《Xevious》第一次要求玩家 同时管空中与地面；1985 年 Konami 的《Gradius》用「能量胶囊」让玩家在推进方向上 **自己选**武器成长；1987 年 Irem 的《R-Type》把蓄力光波、跟随模块和 「背版」式的关卡记忆做到极致。

这十年给我的不是具体机制，是一句话：**卷轴让「空间」从一个平面变成一条走廊，而走廊会自己往你脸上推**。我后来做符箓灵弹那面下压的墙，用的就是这条直觉。

1.1.4 1993 · Batsugun：判定点缩小的那一刻

Toaplan 的《怒首领蜂》（1993）通常被当作弹幕游戏的起点。它做了两件当时很反常的事：**把玩家的判定点缩到飞船的一小块**，以及**在第一关之后铺开越来越几何化的弹型**。子弹不再只是顺便打中你的障碍物，而成了关卡的主体。

> **这一下改变的是「谁在移动」**
>
> 普通射击里，你是**穿过敌人**；弹幕里，你是**穿过图案**。敌人几乎退成了背景，真正要读的是那些粉点和蓝点构成的、会动的谜题，找的就是缝。

紧接着 1994 年 Toaplan 倒闭，旧将另立 Cave，1995 年《DonPachi》补上了**格斗式连段系统**（要求速杀，否则分数掉光），1997 年《DoDonPachi》把公式推到了成熟形态。

1.1.5 1996 – · 东方红魔乡：一个人做出来的流派，和它带来的社群

ZUN 一个人在 PC-98 上开始的《东方 Project》，1997 年的第一作《东方红魔乡》（Story of Eastern Wonderland）就是一部弹幕射击；2002 年的《Embodiment of Scarlet Devil》转到 Windows 发行，才真正把弹幕推向世界。系列后来被吉尼斯世界纪录认定为「最长寿的同人射击系列」。

但东方留给我的不是「弹幕很难」，而是**它周围长出来的东西**：同人音乐、同人画、同人游戏、二创视频。一个单人作品，靠风格一致性，长出了一个比很多大厂作品还大的社群。**风格一致性 > 内容体量**，这是我从东方身上学到的最实用的一条。

1.1.6 2001 起 · 弹幕变成一种可借用的语法

2001 年《斑鸠》让西方玩家第一次正面接触弹幕；2015 年 Toby Fox 的《Undertale》把「弹幕躲避」变成回合制战斗里的一个选项；2016 年《枪火重生》把**智能弹幕**塞进了肉鸽双摇杆；2017 年《尼尔：机械纪元》、2018 年《Deltarune》继续把它接到别的品类上。

2022 年 poncle 的《Vampire Survivors》反过来做了件事：**让玩家成为发射者**，屏幕上全是自己角色的自动攻击。「bullet heaven」就这么成了弹幕的镜像分支。

> **这条血脉留给我的三样东西**
>
> ① **难度应该编码进规律，而不是删掉威胁**；
> ② **判定点要小，但预警要早**。弹幕的公平感来自「我早知道」，不是「弹很少」；
> ③ **风格一致性比内容体量更值钱**。

1.2 肉鸽：从 Rogue 到 Hades

1.2.1 1980 · Rogue：三个词定义了一个流派

加州大学圣克鲁兹分校的学生 Michael Toy 和 Glenn Wichman 做出了《Rogue》，Ken Arnold 也参与了。ASCII 字符、跑在 Unix 小型机上，程序化生成的地下城，死亡即结束。

它定义了三个词，直到今天还在被引用：随机、永久死亡、一局即一生。 第三点最容易被忽略：Rogue 里没有「存档点」这种概念， **你死了就真的没了，你能带走的只有「你学会了什么」**。 这跟「失败后重试」是完全不同的情绪。重试是刷新，Rogue 的死亡是**一次告别**。

它还顺手定义了这个流派的第三个成员：**资源**。 食物、药水、金币都是消耗品，你永远在「现在贪一点」和「留着以后用」之间算账。

1.2.2 1993 · 连流派的名字都是社区投票出来的

「roguelike」这个词来自 1993 年前后的 Usenet 讨论区。当时有 `rec.games.dungeon.`、`rec.games.roguelike.` 等好几个讨论组， 有人提议合并，大家讨论了三个星期，最后选「roguelike」是因为它是「所有选项里最不坏的」。 这个词后来和「Doom clone」平行，都是玩家社区先造词、厂商再追认。

1.2.3 2008 · 柏林解释：定义之争本身就是流派的特征

2008 年柏林国际肉鸽开发大会上，几家社区用 ADOM、Angband、NetHack 等「正典」作品， 起草了一份按价值高低排序的特征清单，这就是**柏林解释**。 高价值特征包括：随机地牢、回合制、网格移动、永久死亡、资源管理、 砍杀战斗、探索与发现、非模态（所有操作随时可用）、复杂度。

清单列出来的目的不是执行，是**划界**。一个流派长到一个流派愿意开会把边界画清楚， 说明它已经成熟了；同样的争论会在后来的「肉鸽 vs 肉鸽 lite」之争里重演一次。

1.2.4 2008 – 2020 · 从 Spelunky 到 Hades：把「随机」从地图挪到玩法

2008 Spelunky（Derek Yu）

程序化生成的洞穴平台跳跃。它证明了**程序生成可以是物理诚实的**：不是随机拼房间，而是随机出一套自洽的规则组合。

2011 以撒的结合（Edmund McMillen & Florian Himsl）

把肉鸽塞进「房间制 + 道具组合」的外壳，证明这套内核在俯视角房间里一样成立。道具之间能不能**互相反应**，成了新的设计题目。

2012 FTL（Subset Games）

把肉鸽的「随机 + 资源」搬进一艘太空船，顺便也把**太空设定**和肉鸽缝在了一起。我做的两条血脉，在这里第一次交汇。

2016 枪火重生（Dodge Roll）

双摇杆肉鸽射击，把弹幕做成了「智能子弹」：子弹会预判你的移动路线，而不是撒满屏。

2020 Hades（Supergiant Games）

拿了 2020 年 TGA 最佳游戏。它做了一件别的肉鸽都没做的事：**把死亡写进叙事**。角色一次次死回冥府，剧情随之推进。失败不再是重来按钮，而是故事的一部分。

Hades 才是肉鸽的真正拐点

在它之前，肉鸽的「失败」是纯粹的**损失**：时间没了、进度没了。 Hades 之后，失败变成了**叙事推进**。 这是一个巨大的设计转向：同一个「你死了」的事件，可以被赋予完全相反的情绪。

1.2.5 2022 · Vampire Survivors：把「随机」推到了极致

它把肉鸽的「每局随机」和弹幕的「铺满屏幕」缝在一起， 但做了一个反直觉的决定：**玩家不再瞄准，角色自动攻击**。 玩家的全部操作是走位与选择升级。 结果是「随机 + 走位 + 堆构筑」这套组合第一次压过了「反应 + 瞄准」。

这条血脉留给我的三样东西

① **一局要自成一段**：有开头、有结尾、有明确的失败时刻；
② **资源必须稀缺**，否则「取舍」这个动词就不存在了；
③ **失败要能带走东西**，哪怕只是一个新的策略想法。

1.3 两条血脉的公共语法，以及我为什么做了六款

把两条血脉摆在一起，会发现它们在讲同一件事，只是用不同的话：

共同主题	太空射击的说法	肉鸽的说法	我落到代码里的形式
难度不该是「变强」	敌人不变强，**弹型更密、但更可读**	敌人不变强，**组合更杂、但每种都认得**	元素反应表：倍率有高低，但每一种都有名字、有形状、有颜色
玩家要多一个**动作**，不是少一个敌人	学会看缝，而不是学躲得更准	学会组合道具，而不是收集更强道具	元素爆发 / 破冰溅射 / 天象预警：都给玩家一次「主动制造」的机会
可读的失败	死前能看出是哪一发打中了你	死前知道自己贪了什么	元素名飘字 + 命中来源；血量、分数、连击全部有明确数字
一局的边界	关卡 / 符卡 / 限时	一次 run / 一条命	3 关 × 8 阵、5 张符卡、一层一房

两边都指向同一条原则：**难度应该是「你需要学会的东西」，而不是「你需要承受的东西」**。 这条原则后来成了我调平衡时的唯一标准（见第五章）。

那么，为什么是六款而不是一款

因为我在试同一个问题的不同答案。**同一题材的两条路线**（符箓灵弹 · 巧阵 / 元素阵） 是同一个设计问题的两个答案。元素反应的价值，到底在「倍率」还是「改变战场元素分布」？ 留两份比只留一份更有说服力。 其余四款则是我把两条血脉分别往极端推的结果：纯粹的射击、纯粹的弹幕、纯粹的肉鸽、纯粹的缝合。

CHAPTER TWO

起点：第一款做砸了

> 所有心路历程最诚实的一版，都是从失败开始讲的。这款的失败不是「不好玩」，而是**「我自己写的判据说它完全没问题」**。

2.1 星际防线 · 初版：我想做什么

第一款是纵版太空射击。目标很朴素：**把三把武器的差异做出来**。单发、双列、激光，让玩家在「点射」「铺面」「穿甲」之间做选择。顺手加了连击倍率、五种道具掉落、每 5 级一个多阶段 BOSS。架构上我做了一个当时觉得挺时髦的决定：**13 个类 + 一个单例 `Game` 对象，所有数值收进 `CFG`**。

这个决定后来被证明是对的（数值集中确实好调），但**它同时给我一种虚假的安全感**：代码看起来那么规整，看起来那么「工程化」，于是我以为它也差不多能玩了。

2.2 「31 项断言全绿」的翻车现场

我给它写了 31 项无头浏览器判据，一跑，全绿。我当时挺得意。然后我去玩，才发现四件事：

① 第 3 关必崩，游戏永远无法通关

生成 BOSS 时写的是 `STAGES[run.stage]`，**少减了一个 1**。于是第 1 关打的是第 2 关的 BOSS，第 3 关取到 `undefined`，读 `.cards` 直接抛 TypeError。
31 项断言全绿都没抓到，因为没有任何一条断言走到「第 3 关的 BOSS」。

② 杂兵从开局到结束一发子弹都打不出来

`mkEnemy` 忘了初始化 `patT`，`e.patT -= dt` 得到 `NaN`，而 `NaN <= 0` 恒为假 → **整个杂兵阶段零威胁**。不报错、不崩溃，游戏照跑，屏幕上就是没人开枪。
于是全部压力都挤到了 BOSS：实测 BOSS 战峰值在场弹 **300 发**。

③ 副武器是主武器的 8.8 倍

`if (P['x']) fireAlt()` 没有射速上限，按住 X 就是每秒 60 发。实测 DPS：主武器 4.08 / 副武器 36.02。**按住一个键就能通关，「主武器」这个设定形同虚设。**
讽刺的是，这个坑是**修另一个 bug 时连带引入的**：在那之前副武器每次按键只打一发，恰好掩盖了「没有上限」这件事。

④ 「出 击」按钮点不到（用户报的 bug）

`html,body{overflow:hidden}` 加上 `.panel` 没有高度上限：1080×520 的窗口里面板高 861px，按钮落在 `top=736`，而面板自身不可滚、文档也不可滚 → **永久够不到，没有任何滚动条能救**。

四个 bug，四种完全不同的失败模式：**下标少减 1、NaN 比较、无上限的重复触发、布局算错**。 而我 31 条断言一条都没碰到它们。

2.3 从这一跤里学到的东西

三句话，构成了后面六款的全部方法论

① **判据不是证据，是工具。**它能证明「这条路径没坏」，但永远不能证明「这条路径存在」。

② **覆盖率的盲区在「边界」**：最后一关、最后一个 BOSS、数组第 0 项、`NaN` 参与比较，断言总是在路径的中间取样。

③ **玩家能看见的东西，判据量不到**。按钮点不到、敌弹不渲染、状态条溢出画布，全都要靠截图。

第三个真 bug 是被**平衡性探针**炸出来的，不是被断言炸出来的。 探针不问「对不对」，只问「数值是不是这个形状」： 比如「玩家钉在原地能活多久」「敌弹密度曲线是什么形状」。 它把「AI 实测第 3 关崩了」这种结果摆在我面前，我才回头去找原因。

从那以后，我每个版本都会问同一个问题

「**如果这套数值是错的，哪条断言会红？**」

答不上来的地方，就是下一次要补的断言。

CHAPTER THREE

骨架：一个刻意的决定

3.1 八层结构

六款游戏的品类差得很远，但骨架几乎一模一样。这不是偷懒，是刻意统一：

层	职责	六款的共同做法
① 舞台	固定逻辑分辨率 + 等比缩放	写死 `W/H`，CSS 只改显示尺寸，坐标永不换算
② 配置	数值集中	调手感只改一处
③ 状态	一局的所有可变数据	统一 `newRun()` 返回一个大对象，重开即重建
④ 输入	键盘 / 指针 / 触屏 → 一个方向量	玩家输入与 AI 输入在 `inp()` 合并
⑤ 时钟	rAF → 归一化 dt → 固定步长	全部 `dt / 16.6667`，再 `clamp(dt, 0, 3)`
⑥ 更新	按固定次序推进子系统	`update(1)` 只接收 1，绝不接收缩放后的 dt
⑦ 渲染	背景 → 实体 → 粒子 → HUD → 屏幕级反馈	层序固定；反馈画在屏幕坐标系
⑧ 探针	把内部状态暴露给无头测试	`__SpaceLine / __TP / __DM / __SK / __SL`

3.2 为什么一定要统一

> **骨架统一 → 问题只会出在各自的「更新层」里**
>
> 这款游戏出 bug 时，我不用从头查整个程序：舞台、时钟、输入、渲染都是验过的，那么问题只可能在**这款游戏特有的那部分逻辑**里。排查范围立刻缩小一半以上。

这套骨架还有一个副作用，我到很后面才意识到它的价值：**它让「AI 接管」可以跨游戏复用**。六个游戏共用同一套「AI 只产出方向向量，再喂给玩家同一套物理」的约定，于是「AI 能通关」这件事在六款游戏里是**同一个命题**，判据可以照抄，bug 也可以照抄（事实上三款游戏共享过一个 rAF 双时钟 bug）。

3.3 一次分歧：同一题材做两条路线

符箓灵弹这个题材我做了**两份**。这不是拖延，是同一个问题的两个答案：

	巧阵	元素阵
操作	左右接球 + 上下调倾角（几何解算）	左右走位 + 上下瞄准（直射）
反应落点	倍率 + 副作用 + **染色连锁**	**范围 / 传播 / 持续 / 状态**
关卡	每 8 阵一章，三种符使轮换	3 关 × 8 阵，固定三阶段符核
额外压力	符矢可击落、元素爆发	护甲块 + **天象**（随机环境元素弹幕）

做第二份的真正动机是我想验证一个命题：**元素反应的价值，到底在「倍率」还是「改变战场的元素分布」？** 答案是**两者都要，而且后者更深**。于是就有了「融化 / 冻结 / 超导」这三条把反应推进到**状态层**的设计。

CHAPTER FOUR

六款，各走一条路

4.1 星际防线 v1 & v2：把参数抽出来，再把三套系统缝起来

v1 的核心收获是那个 13 个类的结构（见第二章）。 v2 是另一件事：**把前三款已经验证过的机制缝进同一套太空射击骨架**，并且证明**缝合的最大风险不是冲突，是「读者看不出哪个系统在起作用」**。

星轨 v2 里同时有：护盾朝向（背击 ×2.5，来自符箓）、小判定点 + 擦弹（来自东方）、 补给舱三选一 + 幸运（来自元气）、翻滚无敌帧（来自元气）。 每条规则我都在 README 里标了来源。

> **缝合时踩到的两个真 bug，都藏在「全绿」后面**
>
> ① **副武器同时绑在 X 键和炸弹上**：按炸弹会顺手打出一发副武器；
> ② **残机归零不结束游戏**：`lives < 0` 应为 `<= 0`， HUD 显示「残 0」却还能一直玩。
> 这两条都不是「数值错了」，而是「**两个系统共用一个键**」和「**边界条件差一个等号**」。

4.2 符箓灵弹 · 巧阵：把「下压」做成可计算的墙

v1.3.0 之前，这一款的形态是「敌人一边下压一边喷满屏符雨」， 玩家的操作只剩左右躲，弹幕一密就变成**看不清、躲不掉、靠运气**。 按 1.1 节的结论：**不可读的威胁再多也只是混乱**。 于是我换了一个问题：

> **把威胁从「看不清的子弹」换成「空间被一点点吃掉」**
>
> 符阵变成一面从上方缓缓压下的**墙**，玩家要做的不再是躲子弹，而是 **在墙落到危险线之前把它敲开**。威胁是可计算的、有面积、可预测的。

这就是弹幕血脉给我的最大一条：**当一个东西让玩家无法判断时，不要减少它，要给它形状**。 墙有面积、有下压速度、有「越线」这个明确失败时刻。 玩家于是从「反应躲避」进入「规划击破」，这才是我想让玩家做的事。

4.3 符箓灵弹 · 元素阵：把反应从倍率推到状态

第二条路线里，我把元素反应拆成三层往上推：

层	形态	载体	玩家怎么利用
第一层	倍率	一张 `RX` 表	换个元素，一发伤害翻倍
第二层	时间 / 空间	燃烧 1.8 秒 DoT + 每 26 帧蔓延 2 块；蒸发 118px 范围打湿	点一排，先不管它，去拆别处
第三层	**状态**	冻结（临时 +5 甲、破冰 1.8 倍 + 溅射）、超导（范围削甲）、融化（熄火 + 打湿）	先挂状态，再开第二枪

第三层是这一版最大的增量。它的核心不是倍率，而是**给玩家一个「两拍」的动作**：先用冰挂住一片，再用火/雷一次性破掉，顺带溅射到邻居。这也带来了这一版最有意思的一个 bug，见 6.3 节。

4.4 星屑符咒：弹幕可读性，判据量不到「东西在哪」

这一款是 1.1 节的结论最直接的一次落地：**难度来自「同时有三处预警」，不来自「看不见的东西突然出现」**。我给所有敌弹加了深色实心描边，并按速度分三档（快弹描边更粗更亮），让玩家一眼能看出「哪颗先打到脸」。

顺带去掉所有 `shadowBlur`。它是全局合成器开销，弹幕一密就是掉帧主因，而**掉帧又会让判定看起来「穿过去了」**。

> **这一款的 AI 是全仓最值得写的部分**
>
> 它不是「躲最近的弹」，而是**解一个两体最近接近点问题**：把玩家速度与弹速度合成相对速度，闭式求出两条匀速直线的最近距离时刻，钳在 30 帧视界内。再在 17 个候选方向里，用字典序 + 自适应容差挑最优。实测 AI 9/9 通关。

4.5 深渊回响：房间即关卡

这一款是肉鸽血脉最直接的落地：**一间房 = 一个 16×10 格的小迷宫**，清完开门，走进去就是下一间。没有走廊、没有分支、没有 BFS。

但它也是「缺陷档案」里最难看的一页。解析代码时对着「定义 → 调用」做交叉核对，找到**三处逻辑在跑、但玩家感知不到的缺口**：

- `drawEBullets()` 定义了，但 `draw()` 的调用表里没有它：**敌弹真实扣血，屏幕上什么都没有**；
- `pl.rx/ry` 赋值后从不清零：**翻滚时几乎不撞墙**，且翻滚结束后探针方向永远停在最后一次翻滚；
- `b.delay` 全文件没有递减语句，**法师「三连点」实际只打出 1 发**。

> **「定义了却没人调用」和「字段只写不读」，是静态检查抓不到的一类 bug**
>
> 语法通过、判据全绿、页面能玩，但机制**衰减或消失**。可行的自动检查是：对每个 `function drawXxx` 断言它在 `draw()` 里被引用一次；对每个「有 `delay:` 赋值」的字段断言存在递减语句。

4.6 星轨 v2：缝合而不打架

缝合的最大风险是**读者看不出哪个系统在起作用**。我的处理办法是给每条规则标来源，并在 BOSS 弹型上做数据驱动（`cards[].pattern`），而不是 `if (stage === 4)`。结果是 3 关扩到 5 关、BOSS 弹型 3 种扩到 5 种，代码没有变成 if 汤。

CHAPTER FIVE

难度观：我怎么从「削敌人」转向「加解法」

5.1 v1.4.0：一次统一降难度

v1.4.0 我给六款统一降了难度，方法是「缓曲线」：把 `a + b·n` 里的 **斜率 b 调小**，前几关的绝对难度几乎不动，后期才是原本劝退的地方。 命 3→5、判定 3.5→2.8px、擦弹 24→30、残机 3→4……

这一版的信条写在文档里：**让玩家多一个可执行的动作，而不是让敌人少一个动作。**

5.2 v1.5.0：降过头了，同一条曲线不能跨品类照搬

元素阵是并行并入的，也按同一条「降难度」口径压了一档： 心 4→5、墙速 `10.5+0.55n → 8.8+0.36n`、符块血 `10+2n → 9+1.5n`。 用户实测回来的反馈是三条：**射速过快、墙移动太慢、血量太低**。

> **为什么别的五款降了没事，这一款降过头了**
>
> 因为**它们的难度模型不一样**。另外五款的压力来自敌人数量与弹幕密度； 元素阵的压力来自**「元素反应是否构成硬约束」**。 而符块血压到 9 之后，直接伤害是 12，**无反应也一发一块**， 护甲和元素反应一起失效，这条硬约束断了。
>
> 同一套「缓曲线」手法，用在那五款上是对的，用在这款上是**把截距压到了机制阈值以下**。

修正的方向也因此很清楚：**不是把曲线调回去，是把机制调回来**。 符块血 `9+1.5n → 14+2.2n`（大于直接伤害 12）， 于是「≥1.3 倍反应才能一发破」，反应从「打得更快」变回「**必要**」。

5.3 最后我信什么

做法	本质	本项目的实例
加解法	玩家多一个按钮	元素爆发、灵弹可击落符矢、越线后墙回退 84px、破冰溅射
给容错	同样的失误不再致命	命 3→5、符盘 52→58、判定 3.5→2.8、擦弹 24→30、残机 3→4、玩家血 3→4
缓曲线	前几关几乎不变，后期才明显缓	墙速 `min(13, 7.6+0.32n)`、符块血 `8+1.3n`、升级 16→22 秒 / 上限 12→10

> **补上第五条，也是最重要的一条**
>
> **先确认这条品类的「机制阈值」在哪，再谈降难度。** 难度是相对于机制说的。同一个数值在「靠数量施压」的游戏里是宽松， 在「靠机制施压」的游戏里可能是失效。

CHAPTER SIX

AI 观：让机器替我玩

6.1 为什么 AI 接管是「诚实的验收」

六款游戏都有 AI 接管（快捷键 `E`，或面板上的按钮）。 做 AI 接管，是因为**「数值调得对不对」这件事，只有让机器真的去打才能回答**。

判据可以断言「这一发伤害是 12」，但断言不了「玩家能不能在第 3 关活下来」。 AI 接管把这个问题变成了一个**可复现的统计量**： 「AI 连续 5 轮各 90 秒，每轮破阵 ⩾ 2」。

> **AI 接管的验收价值**
>
> 它把「好玩」这个主观判断，换成了一个可以天天跑的数字： 平均在场弹 85.5 → 17.3，峰值 300 → 39；第 1 关符卡从「打满 45 分钟都死不了」变成 633 帧打掉。 这种句子没法靠感觉写出来。

6.2 AI 写不对的三种方式

作弊方式	怎么抓
改判定点、加无敌帧	开关 AI 前后逐个比对物理参数快照，必须逐个相同
瞬移 / 穿墙	连续推进 N 帧，量单帧位移不得超过玩家移速（实测 ⩽ 7.8px）
预知未来	AI 只允许读**当前**的位置与速度；威胁表用的相对速度是解析解，不是「下一帧它会到哪」

星轨 v2 的 AI 接管最费劲，因为它得在三种来源的威胁里同时活下来： 护盾朝向决定伤害倍率、擦弹要主动去蹭、补给舱要主动去吃。 最后我给它定的规矩是：**AI 不作弊，只产出方向向量，喂给玩家同一套 `movePlayer` 物理**。 这样「AI 走得通」=「这套数值真的能打」，而不是「AI 用了外挂所以能赢」。

6.3 v1.5.1：AI 看不见新元素，比「AI 打不赢」更隐蔽的失败

补录元素阵的演示视频时，我注意到一件怪事：**AI 接管下，冰一次都没被选中过**。

```
// 旧写法：取「第一个倍率 ≥1.3 的元素」
good = EL_ORDER.find(e => {
  const r = reactionOf(e, def);
  return r && r.mul >= 1.3 && r.kind !== 'burn';
}) || 'anemo';
```

这是顺序，不是强弱。冰的三条反应单发倍率都不占优 （融化 2.0 与蒸发 2.0 打平、冻结 1.0、超导 1.4 低于超载 1.5）， 而 `EL_ORDER` 里冰排在火水雷风之后， `find` 永远轮不到它。

这不是「AI 打不赢」，是「新加的元素在 AI 眼里不存在」

玩家手动按 `5` 能用，AI 一次都不会。 而 31 项判据**全绿**，因为没有一条断言检查「AI 会不会用新元素」。

根因和第一章那条血脉有关：**冰的收益要两发才兑现** （冻结成片 → 火破冰 1.8 倍 + 溅射；超导 → 范围削甲 + 半伤溅射）， 而倍率表是**单发**量表。修法是给冰单独记一个「组合价值」，再和单发倍率取大值比较。 改完之后，AI 会自动打出「**冰 → 冻结 → 火破冰 1.8 倍 + 溅射**」这条二段循环。 也就是 v1.5.0 最想让人玩到的那个动作。

这一条是我在整个项目里学到最有用的一句话

加新机制的时候，要同时问一句「AI 能不能用上它」。

玩家能按出来 ≠ 系统能用起来。前者是**入口**，后者是**用得起来**。 一个只有入口、走不通全链的新机制，在自动游玩和自动验收里等于不存在。

CHAPTER SEVEN

验证观：全绿不等于对

7.1 判据自己错的十一种方式

这个项目最不像「个人练习作」的地方，是它有一套能自己给出结论的验证体系：**247 项断言、约 40 秒跑完、六款全绿**。但更值得写下来的是**判据自己错**的那十一次。

变种	症状	真因
在 `start()` 前取 `run` 引用	16 条同时红，全是「什么都没发生」	`start()` 内 `newRun()` 重建了对象，判据改的是上一局的尸体
被测对象摆在屏幕外	读数恒为 0	BOSS 符矢在 `wallY=-260` 生成即被出界过滤
假设实现内部的 guard 上限	`tick(10)` 只推进 8 步	判据不该依赖被测代码的保护值
1× / 2× 期望值写反	「两倍速比常速慢」	标签和断言对不上
统计断言时间预算不足	AI 破阵 0	实测 AI 在 53 秒时已破 13 块，只是给的窗口太短
全仓改 HiDPI 后像素采样偏	读到的永远是背景色	`getImageData` 读设备像素，采样坐标要乘 `getTransform().a`
在 `destroyBlock` 之后读 `run.boss`	恒为 `false`	击破时副作用会把它置空，取值顺序必须在副作用之前
用「单层总血量」比难度	样本 24 次，但结论是倒的	敌人种类随机：第 6 层抽到石像 13.5 血、第 12 层全是小怪 6.3 血 ×8。改成按 `kind` 分桶比同类型
全文正则扫特征串	「旧 shadowBlur 还在」误报	命中的是**注释**和别的类。只扫目标段落，且只认 `ctx.shadowBlur` 这种真实调用
带横漂 / 重力的弹，隔 N 帧再数	水弹分裂判据报「小水滴 0 滴」	水滴会被回收或出界，`step(120)` 只能看到「剩下的」。**必须在命中当帧抓**
比较里带可能缺失的字段	燃烧 DoT 判据掉 0 血	判据造的块没给 `frozen` 字段，而代码写 `b.frozen <= 0`，`undefined <= 0` 恒 false，分支静默不进

7.2 「定义了却没人调用」和「字段只写不读」

上一节列的十一种，是「判据本身写错了」。还有一类更隐蔽：**判据是对的，但被测代码有洞，而洞不在任何一条断言的路径上**。深渊回响那三处就属于这一类，见 4.5 节。

7·3 截图是判据的一部分

五条「v1.4.0 新抓到的缺陷」里，**三条是截图发现的**：状态条溢出画布、房间长期偏紫红、符卡名字被符块盖住。这不是判据不够多，而是判据量的是**状态**（数值、标志位），量不到**可见性**（东西在哪、字有没有出界、颜色有没有盖住）。

> **所以每轮改完的固定动作是三步**
>
> 跑判据 → 截图 → **人眼看**。
> 符箓灵弹的「墙生成在屏幕外，每阵开头 5 秒空场」、深渊回响的「敌弹从不渲染」，都是判据抓不到、肉眼一眼就能看出来的。

CHAPTER EIGHT

借鉴来源与取舍

六款游戏各自站着一个明确的参照系。逐条写清**借了什么、改了什么、没碰什么**。

作品	借了什么	改了什么	没碰什么
《原神》限时玩法「符箓灵弹」	元素反应表的概念：火水雷风相克，蒸发 / 超载 / 感电 / 扩散四类反应	把「倍率」改造成**范围 / 传播 / 持续 / 状态**四层；倍率全部为本作节奏重排	任何美术、角色、图标、UI、字体；没有角色，没有抽卡，没有数值表照搬
《东方 Project》	弹色即语义、判定点极小、擦弹加分、符卡限时	加了 10 张符卡与关卡链；AI 威胁表用**相对速度解析解**而非最近弹	任何立绘、背景、编曲、音效；没有自机、擦弹系统、Spell Card 名称
《元气骑士》	「一房一清 → 开门 → 下一层」的房间循环；翻滚无敌帧；三选一升级	把 16 格的迷宫换成**同尺寸房间 + 瞬移重建**，没有走廊没有分支；怪物血量成长重算	武器系统、角色、宠物、联机、任何素材
《Rogue》/ 肉鸽	随机、永久死亡、一局即一生、资源稀缺	把「随机地图」换成**随机化参数**（速度 / 血量 / 混色概率），因为单文件体积做不出真地图	回合制、网格移动、装备耐久、食水。这些是传统 roguelike 的正典特征，但会毁掉实时手感
《R-Type》 / 《Gradius》	卷轴带来的「空间被推着走」；武器成长的取舍感	卷轴换成**下压的符阵墙**；武器成长换成元素反应	关卡设计、音乐、任何美术
《雷电》《斑鸠》《怒首领蜂》	弹幕的「几何可读」原则：密度可以高，但图案必须能看出缝	把弹幕搬到**竖版房间制**并与肉鸽融合；给敌弹加深色描边与速度分档	任何关卡、任何美术、任何音频

必须说清的取舍

上面这组倍率是为本作节奏重排的，**不是原作的官方数值**；原作里超载 / 感电 / 扩散是固定伤害，这里改成倍率才能和符块血量配合。参考的只是公开资料里「谁触发谁、谁强谁弱」的结构。

另外：**本项目没有使用任何第三方模型、贴图、字体或音效素材**，画面全部是 Canvas 2D 程序化绘制。

CHAPTER NINE

交付与运行

9.1 怎么玩

每款游戏就是一个 `index.html`，**双击即玩**：不需要服务器、不联网、没有第三方库。

```
cd games
npx serve .        # 个别浏览器对本地 file:// 有限制时，起个静态服务器
# 打开 http://localhost:3000/
```

游戏	移动	开火	其它
星际防线 · 初版	← → ↑ ↓ / WASD	空格	AI E · 二倍速 T
符箓灵弹 · 巧阵	同上	空格	1 – 4 / Q 换元素
符箓灵弹 · 元素阵	← → 走位 ↑ ↓ 瞄准	空格（带元素）	1 – 5 / Q 换元素（火水雷风冰） · P 暂停
星屑符咒 · 弹幕手记	同上	Z	Shift 精确低速 · X 炸弹
深渊回响	同上	J（自动瞄准）	Shift 翻滚 · E 开箱 · 1/2/3 选升级
星轨防线 v2	同上	Z 主武器 · X 副武器	C 炸弹 · Shift 翻滚 / 精确

9.2 怎么跑判据

```
cd games
node _run_all.js                                    # 六款全跑，约 40 秒
node talisman-array/verify/smoke.js index.html __TP # 单款
```

判据跑在**真实浏览器**里（headless Edge + 原生 WebSocket 直连 CDP，不引 puppeteer），不是 Node 里搭 DOM stub。Canvas 的实际绘制路径（渐变、`shadowBlur`、`ellipse` 兼容）只有真浏览器能暴露。

9.3 怎么录演示

```
node _tools/record-demo.cjs --game talisman-array --out "06_符箓灵弹-元素阵.mp4"
```

冻结 rAF 后按固定步长推进、逐帧抓画布像素、交给 ffmpeg 压 H.264。所以同一份代码每次录出来的画面**完全一致**，返工只要重跑脚本而不用重录。

9.4 仓库结构

```
games/
  index.html                ← 版本档案首页（由 _build_site.js 生成，别手改）
  manifest.json             ← 元数据与全部版本记录
  _run_all.js               ← 一键并行回归
  _snapshot.js              ← 留档到 versions/<日期>/
  _build_site.js            ← 用 manifest + _logs 生成首页
  _tools/record-demo.cjs    ← 演示录制器
  docs/                     ← 本文档 + 实现解析 + 交付说明 + PDF
  demo/                     ← 六段实机录屏 + 演示合集页
  <游戏目录>/index.html      ← 唯一源文件
  <游戏目录>/verify/smoke.js ← 冒烟判据
  versions/<日期>/           ← 历史快照，只增不改
```

APPENDIX A

技术解析摘要

完整版见 `docs/实现解析.html`。这里只列六款共用的骨架与最值得一读的三段实现。

A.1 八层骨架（详见第三章）

A.2 符箓灵弹 · 巧阵：把「打中球」变成可解的方程

```
const ang = Math.atan2(blockCY(tgt) - b.y, blockCX(tgt) - b.x);
wantTilt = clamp(clamp(ang, -Math.PI + 0.4, -0.4) + Math.PI / 2, -TILT_MAX, TILT_MAX);
```

三段式：① 接球走位 ② 球接近拍面时按「目标块得分」反解倾角 （得分 = 倍率×46 − 高度×0.055 − 水平距离×0.045 − 血量×0.55）③ 每 20 帧换一次元素。 **AI 做的也只有这三件事**，而且只写方向键。

A.3 星屑符咒：威胁表的解析解

```
const rx = b.x - px, ry = b.y - py;
const rvx = pvx - b.vx,   rvy = pvy - b.vy;
const rr = rvx*rvx + rvy*rvy;
let tt = rr > 1e-6 ? -(rx*rvx + ry*rvy) / rr : 0;
tt = tt < 0 ? 0 : (tt > T.look ? T.look : T.look);   // look = 30 帧
const d = Math.hypot(rx + rvx*tt, ry + rvy*tt);
```

这是**闭式解**而非采样：把玩家速度与弹速度合成相对速度， 直接求两条匀速直线的最近距离时刻 `t* = -(r·v)/|v|²`，钳到 30 帧视界内。 之后在 17 个候选方向里，先按全局最优余量设自适应容差（12 / 6 / 2），只让接近最优的方向进入第二轮用目标点积排序。 这样「为了瞄准站着不动」不会因为目标权重大而反噬安全性。

A.4 深渊回响：同帧双轴施加，再按轴回退

```
pl.x = clamp(pl.x, 24, W - 24);
pl.y = clamp(pl.y, 24, H - 24);
if (!isFloor(pl.x + Math.sign(pl.rx || ax || 0) * pl.r, pl.y)) pl.x -= ax * pl.spd * dt;
if (!isFloor(pl.x, pl.y + Math.sign(pl.ry || ay || 0) * pl.r)) pl.y -= ay * pl.spd * dt;
```

不是经典的「先移 X 再解 X、再移 Y 再解 Y」，而是同帧双轴先施加、 再逐轴检查前缘采样点（当前坐标 + 朝向 × 半径）。前缘不在可走格就把该轴这一帧的位移撤回。

APPENDIX B

文件地图

路径	规模	作用
`games/space-line-v1/index.html`	5,649 行 / 236 KB	初版存档：13 个类 + CFG 二十一组配置，源码原样保留
`games/talisman-pinball/index.html`	2,056 行 / 84 KB	符箓灵弹 · 巧阵：接球 + 倾角反解 + 染色连锁 + 章节 BOSS
`games/talisman-array/index.html`	2,204 行 / 92 KB	符箓灵弹 · 元素阵：五元素 + 三层反应 + 天象 + AI 组合价值
`games/touhou-danmaku/index.html`	2,610 行 / 120 KB	星屑符咒：10 张符卡 + 解析威胁表 + 擦弹
`games/soul-knight/index.html`	2,339 行 / 100 KB	深渊回响：房间循环 + 三选一升级 + 分层 BOSS
`games/starlane-v2/index.html`	2,640 行 / 125 KB	星轨防线 v2：缝合三套系统的太空射击
`games/*/verify/smoke.js`	合计 247 项断言	每款的无头回归判据
`games/_logs/`	6 份	每次回归的原始输出，首页断言数从这里读
`games/versions/<日期>/`	6 个快照	每次发布的源码 + 判据 + 日志
`games/docs/`	4 份	本文件 · 实现解析 · 交付说明 · PDF
`games/demo/`	6 段 mp4	实机录屏 + 演示合集页

APPENDIX C

版本时间线

1.0.0 2026-10-12 · 第一个留档版本

四款游戏全部跑到判据全绿，做了一轮跨游戏缺陷扫描与修缮。此前 `games/` 不在版本控制里，更早的状态没有快照可回填。

1.1.0 2026-10-13 · 平衡探针炸出三个真 bug

加了 AI 接管与二倍速；修掉用户报的「出击按钮点不到」布局 bug；补录了会话最初那一版。判据从 31 项涨到 105 项，但最有价值的产出不是新功能，是探针挖出的三个真 bug，它们此前全都藏在「31 项断言全绿」后面。

1.2.0 2026-10-13 · 探针第三代

先修一个影响全部 5 款判据脚本的收尾 bug（每次 smoke 白等 3~5 分钟），再用平衡探针第三代逐条验证上一轮的下调成果。结果三条数据全部不可信、查清后有一个真 bug、一次平衡改动过头、三条判据自己有缺陷。

1.2.1 2026-10-15 · 深色 → 浅色

档案页从深蓝底改成浅色。触发点是「深色模式不好看」，但真正花时间的地方不是换背景色。是深色主题那套亮色强调色在白底上等于看不见，必须整组重挑。顺手还揪出两个只在成品里才看得见的产物级 bug。

1.3.0 2026-10-19 · 八条整改

去掉屏幕抖动与全屏白闪；把符箓灵弹整个重做成「下压的元素符阵墙 + 四元素反应」；给所有游戏补齐 AI 接管与二倍速；让星屑符咒的弹幕变成可通关的；让深渊回响一关之后继续下潜。判据 120 → 183 项。

1.4.0 2026-10-20 · 内容做厚、难度做顺

符箓灵弹的风拆成三条分支并加入反应链与元素爆发，章节化后每 8 阵一个三种弹型的符使 BOSS；五款全部下调难度并补 BOSS / 关卡；四款敌弹统一描边分级；顺手修掉深渊回响三处「玩家感知不到」的实现缺口；并入并行实现的「符箓灵弹 · 元素阵」。判据 183 → 233 项。

1.5.0 2026-10-22 · 降难度降过头了，反向调

用户实测反馈「射速过快、墙太慢、血量太低」。上一轮并档时按降难度口径压过头了。这轮三项数值反向调整，并新增「天象」环境元素弹幕与冰元素及其三条反应。判据 233 → 246 项。

1.5.1 2026-11-10 · AI 看不见新元素

补录演示时发现 AI 永远不会选冰（`find` 取的是顺序不是强弱），修掉，并补一条判据钉住。同时把演示与文档搬进仓库：新增交付说明页、六段实机录屏、开发过程 PDF，档案页加「文档与演示」入口。判据 246 → 247 项。

如果这份文档只让你记住一件事

先确认这条品类的「机制阈值」在哪，再谈降难度；先确认新机制在 AI 眼里存在，再谈它好不好玩。

这两条都不是从设计书里学来的，是被自己的判据打脸两次之后才学会的。

APPENDIX D

参考资料与延伸观看

> 第一章和第二章提到的每一个年份、开发者与机制，都不是凭印象写的。 下面把出处逐条列出来，链接在 2026-11-10 逐条发过 HTTP 请求确认可达。

关于这批链接，有一条踩过的坑值得记下来

GDC Vault 的 URL 形如 `gdcvault.com/play/ID/任意标题`， **后半截标题随便写都返回 200**。真正的内容只看 ID。 我按「地址能打开」筛了一遍，结果有两条打不开的其实能开，只是标题是别的演讲。 所以下面每个 GDC 链接都是**按页面标题逐条核对**的，宁可少收也不放假出处。

D.1　太空射击：作品与出处

作品	年份	文中位置	出处
Spacewar!	1962	1.1 起点	Wikipedia
Space Invaders	1978	1.1.1 三条语法	Wikipedia
Galaga	1981	1.1.2 编队与俯冲	Wikipedia
Defender	1981	1.1.3 横向卷轴与保护人类	Wikipedia
Scramble	1981	1.1.3 燃料与地形	Wikipedia
Xevious	1982	1.1.3 地面目标	Wikipedia
Gradius	1985	1.1.3 能量胶囊与自选成长	Wikipedia
R-Type	1987	1.1.3 蓄力光波与背版	Wikipedia
Toaplan	—	1.1.4 弹幕的诞生地	Wikipedia
Batsugun	1993	1.1.4 判定点缩小	Wikipedia
Cave	—	1.1.4 Toaplan 之后的继承者	Wikipedia
DonPachi	1995	1.1.4 格斗式连段	Wikipedia
DoDonPachi	1997	1.1.4 公式成熟	Wikipedia
东方 Project	1996 –	1.1.5 一人做出来的流派	Wikipedia · 官网
红魔乡（Scarlet Devil）	2002	1.1.5 转 Windows 出圈	Wikipedia
ZUN	—	1.1.5 东方作者	Wikipedia
斑鸠	2001	1.1.6 西方第一次正面接触	Wikipedia
Undertale	2015	1.1.6 弹幕进回合制	Wikipedia
枪火重生	2016	1.1.6 智能子弹	Wikipedia
Vampire Survivors	2022	1.1.6 反向弹幕	Wikipedia
弹幕（Danmaku / Bullet hell）	—	1.1.4 词条	Wikipedia
Shoot 'em up	—	1.1 全章总览	Wikipedia

D.2　肉鸽：作品与出处

作品 / 事件	年份	文中位置	出处
Rogue	1980	1.2.1 三个词	Wikipedia
神秘迷宫（Mystery Dungeon）	1995 –	1.2.1 日本分支	Wikipedia
「roguelike」这个词	1993	1.2.2 Usenet 社区投票	Wikipedia
柏林解释	2008	1.2.3 划界	原文（RogueBasin）
Spelunky	2008	1.2.4 程序生成可以是物理诚实的	Wikipedia · Derek Yu
以撒的结合	2011	1.2.4 房间制 + 道具组合	Wikipedia
FTL	2012	1.2.4 两条血脉第一次交汇	Wikipedia
Hades	2020	1.2.4 把死亡写进叙事	Wikipedia
Diablo	1996	1.2.3 常被划到边界外	Wikipedia

D.3　视频与讲座

内容	形式	说明
Breathing Life into Greek Myth: The Dialogue of 'Hades'	GDC Vault 演讲（视频 + 幻灯片）	Supergiant 的 Greg Kasavin 与 Darren Korb 拆解 22000 句配音怎么做出来的，以及 Early Access 怎么反过来帮了叙事。页面标题已逐字核对
GDC 2018 免费演讲目录	GDC Vault 索引	想找「肉鸽 × 叙事」那一年的演讲，从这里翻
Roguelikes And Narrative Design With Hades	GDC 播客（音频 + 逐字稿）	Kasavin 讲「叙事连续性」：肉鸽结构里 NPC 记得你上一局做了什么
GDC plays Hades with Greg Kasavin	GDC 访谈（音频 + 逐字稿）	边打边讲：为什么 Early Access 适合故事多的游戏
Watch Greg Kasavin discuss the development of Hades	开发者访谈	2018 年 TGA 宣布之后的即时访谈
GDC 2006 实验游戏工作坊纪要	演讲纪要（文字）	含 Jonathan Mak 讲《We Built This Shooter On Rock and Roll》：他怎么把射击做成一张「专辑」，每关对应一段乐曲结构，弹型对应段落
GDC（YouTube）	视频频道	GDC 官方频道，公开演讲视频的稳定入口
Retro Game Reviews（YouTube）	视频频道	逐款讲街机原作的长视频：Space Invaders、Galaga、R-Type、Gradius 各有专集。看机制怎么被一步步改出来的，比看史实文字直观
Game Maker's Toolkit（YouTube）	视频频道	拆解单个设计机制怎么运作；肉鸽、随机生成、难度曲线这类题目上有几期和本文的论点直接相关
TotalBiscuit（YouTube）	视频频道	老牌的「游戏设计文档」系列，其中玩 Rogue 的那几期是理解这个流派的入门片

D.4　延伸阅读与查证工具

资源	用途
A Brief History of Shoot-'Em-Ups: From Space Invaders to Touhou	一篇长文把固定屏、卷轴、弹幕三段脉络串起来，本文第一章的骨架主要来自它
The International Arcade Museum（KLOV）	街机原作的资料库：机型、年份、厂商、玩法说明，查早期街机机种最靠谱
Guinness：最长寿的同人射击系列	东方系列的吉尼斯记录，1.1.5 提到的那个说法的原始出处
东方 Project 官网	官方作品列表与发行顺序
Roguelike 词条	1993 年 Usenet 命名、柏林解释、roguelite 之分的最完整梳理
柏林解释原文	2008 年那次会议的原始文本，含正典作品与高/低价值特征清单

最后一句关于「出处」的话

上面这些链接是**2026-11-10 逐条发 HTTP 请求确认可达**的，不是抄来的。 写这份文档时我改了一个更老的教训：Wiki 条目会变、页面会挪， 所以「链接能打开」只是最低门槛，**链接打开之后是不是我要说的那一页**，得逐条看标题。 GDC Vault 那两条就是被这条门槛拦下来的：打开是 200，标题却是别的演讲。